

Deploying SokoPay to a DigitalOcean droplet (pilot test)

This guide sets up the whole SokoPay platform on **one DigitalOcean droplet**, running the way it will in production, so the apps and portals can be tested end to end. When testing is finished, section 11 wipes everything.

The stack:

- **PostgreSQL 16**, the production database. The local demo's SQLite file is only a laptop convenience.
- **Redis**, for the cache and the task queue.
- **Celery** workers and the scheduler.
- **Caddy**, which gets and renews the HTTPS certificate automatically.
- **Docker Compose**, which runs it all.

Test data only. The server runs the `staging` profile:

- **Same as production:** HTTPS only, secure cookies, the encryption key required, PostgreSQL, JSON logs, debug off.
- **One difference:** it runs against the **mock payment partner** until Korba or Nsano is contracted and certified.

Every web page shows a "TEST ENVIRONMENT" banner. Do **not** enter real customers' data or real Ghana Card numbers. The droplet sits outside Ghana, which is fine for demo data but not for real customer data without Bank of Ghana and Data Protection Commission approval.

1. What you need

Item	Notes
DigitalOcean account	With billing set up.
A domain name, or none	Example: <code>test.sokopay.com.gh</code> . With no domain, use the free <code>sslip.io</code> name (section 4). The phone apps need real HTTPS, which Caddy provides automatically.
An SSH key on your computer	Create one in PowerShell: <code>ssh-keygen -t ed25519</code> .
This repository on your computer	The <code>sokopay-platform</code> folder.
Git Bash	Comes with Git for Windows. Used for <code>scp</code> and the packaging script.

Droplet size

Use	Plan	Approx. monthly price
Small test (a handful of testers)	Basic Regular, 2 vCPU / 4 GB RAM / 80 GB	\$24
Larger pilot	Basic Premium, 4 vCPU / 8 GB	\$56

Region: London (LON1) or Frankfurt (FRA1), the closest to Ghana. DigitalOcean has no African region.

Image: Ubuntu 24.04 LTS.

2. Create the droplet

1. DigitalOcean control panel: **Create**, then **Droplets**.
2. Region **London**, image **Ubuntu 24.04 (LTS) x64**, size from the table above.
3. **Authentication**: choose **SSH Key**, then **New SSH Key**, and paste the contents of `C:\Users\
<you>\.ssh\id_ed25519.pub`. Don't use a root password.
4. Tick **Monitoring** (free CPU, memory and disk graphs and alerts). **Backups** are optional for a short test, because the database is also backed up by script (section 8).
5. Hostname: `sokopay-test`, then **Create Droplet**. Note its **public IPv4**, for example `203.0.113.10`.

2.1 Cloud firewall

Networking, then **Firewalls**, then **Create Firewall**, named `sokopay-test`:

Inbound rule	Port	Sources
SSH	22	Your own IP only (look it up at https://ifconfig.me)
HTTP	80	All IPv4 and IPv6 (needed for the certificate)
HTTPS	443	All IPv4 and IPv6

Leave outbound as "all". Apply the firewall to the droplet. PostgreSQL and Redis are never exposed: they only listen inside Docker's private network.

3. First login and server hardening

From Git Bash or PowerShell:

```
ssh root@203.0.113.10
```

On the droplet, run these as root:

```
# Updates and automatic security patches
apt update && apt -y upgrade
apt -y install ufw fail2ban unattended-upgrades
dpkg-reconfigure -f noninteractive unattended-upgrades

# A non-root user for everything else
adduser --disabled-password --gecos "" deploy
usermod -aG sudo deploy
mkdir -p /home/deploy/.ssh && cp ~/.ssh/authorized_keys /home/deploy/.ssh/
chown -R deploy:deploy /home/deploy/.ssh && chmod 700 /home/deploy/.ssh
echo "deploy ALL=(ALL) NOPASSWD:ALL" > /etc/sudoers.d/deploy

# SSH: keys only, no root login
sed -i 's/^#\?PermitRootLogin .*/PermitRootLogin no/; s/^#\?PasswordAuthentication .*/PasswordAuthentication no/' /etc/ssh/sshd_config
systemctl restart ssh

# Host firewall (in addition to the cloud firewall)
ufw allow OpenSSH && ufw allow 80/tcp && ufw allow 443/tcp && ufw --force enable

# 2 GB swap (helps the image build on a 4 GB droplet)
fallocate -l 2G /swapfile && chmod 600 /swapfile && mkswap /swapfile && swapon /swapfile
echo '/swapfile none swap sw 0 0' >> /etc/fstab

# Time zone for logs
timedatectl set-timezone Africa/Accra
```

Before closing the root session, open a **new** terminal and check you can log in as `deploy` :

```
ssh deploy@203.0.113.10
```

3.1 Install Docker

As `deploy` :

```
curl -fsSL https://get.docker.com -o get-docker.sh && sudo sh get-docker.sh && rm get-docker.sh
sudo usermod -aG docker deploy
exit
```

Log in again so the group change applies, then check: `docker compose version` .

4. Domain name (DNS)

With your own domain: at your DNS provider (or in DigitalOcean **Networking**, then **Domains**), add an **A record**. For example, `test.sokopay.com.gh` points to `203.0.113.10` . Wait until `ping test.sokopay.com.gh` shows the droplet's IP.

With no domain: use `203-0-113-10.sslip.io` (the droplet IP with dashes, plus `.sslip.io`). It already resolves to your droplet, and Let's Encrypt issues certificates for it. Use that name everywhere this guide says `test.sokopay.example` .

5. Upload the code

The repository is not on GitHub, by your instruction, so the code is copied straight from your computer.

On your computer (Git Bash, in the `sokopay-platform` folder):

```
bash deploy/scripts/package.sh
ssh deploy@203.0.113.10 "sudo mkdir -p /opt/sokopay && sudo chown deploy:deploy /opt/sokopay"
scp sokopay-deploy.tgz deploy@203.0.113.10:/opt/sokopay/
```

The package (about 0.5 MB) contains only `backend/` and `deploy/`. It leaves out your virtual environment, local databases, `.env` files and the demo logins.

On the droplet:

```
cd /opt/sokopay && tar xzf sokopay-deploy.tgz && rm sokopay-deploy.tgz
ls          # backend  deploy
```

6. The `.env` file (settings and secrets)

All configuration lives in **one file on the server**: `/opt/sokopay/deploy/.env`. Both Docker Compose and the app containers read it.

- Never commit it, email it or paste it into chat.
- Its permissions are `600`, so only the `deploy` user can read it.

6.1 Create it and generate the secrets

```
cd /opt/sokopay/deploy
bash scripts/make-secrets.sh
```

This copies `.env.example` to `.env`. It then fills every `CHANGE_ME` with a strong random value and puts the same passwords into the database and Redis URLs. The values it generates:

Variable	What it is
<code>SECRET_KEY</code>	Django signing key (sessions, tokens, signed links)
<code>FIELD_ENCRYPTION_KEY</code>	Encrypts Ghana Card numbers. If you lose it, that data can't be read.
<code>POSTGRES_PASSWORD</code>	Database password (also in <code>DATABASE_URL</code>)
<code>REDIS_PASSWORD</code>	Redis password (also in <code>REDIS_URL</code>)
<code>USSD_SHARED_SECRET</code>	The USSD gateway must send this (section 10.4)
<code>RAIL_WEBHOOK_SECRET</code>	Signs the mock partner's callbacks

6.2 Fill in the domain

```
nano .env
```

Change these five lines to your domain:

```
SOKOPAY_DOMAIN=test.sokopay.com.gh
ACME_EMAIL=you@yourcompany.com
ALLOWED_HOSTS=test.sokopay.com.gh,localhost,127.0.0.1
CSRF_TRUSTED_ORIGINS=https://test.sokopay.com.gh
QR_BASE_URL=https://test.sokopay.com.gh
```

Save with Ctrl+O, then exit with Ctrl+X. Check that nothing is left unfilled; the following should print nothing:

```
grep CHANGE_ME .env
```

6.3 What the other settings mean

Variable	Pilot value	Notes
DJANGO_SETTINGS_MODULE	config.settings.staging	Production hardening plus the mock partner. Real production uses <code>config.settings.prod</code> (section 12).
COMPOSE_PROFILES	localdb	PostgreSQL runs in Docker on the droplet. Leave it empty if you use DigitalOcean Managed PostgreSQL (section 6.4).
SOKOPAY_ACTIVE_LICENCE	DEMI	Every feature on, for a full test.
RAIL_PROVIDER , KYC_IDENTITY_PROVIDER	mock	No real money or real identity checks.
SMS_PROVIDER	console	Sign-in codes are written to the log (section 10.2). Set to <code>hubtel</code> with credentials for real SMS.
PUSH_PROVIDER	console	Set to <code>fcm</code> once Firebase is configured.
SENTRY_DSN	empty	Optional error tracking.

6.4 Optional: DigitalOcean Managed PostgreSQL instead

This is closer to real production, with automatic backups and failover. It costs about \$15 a month extra.

1. **Databases**, then **Create Database Cluster**: PostgreSQL 16, same region, smallest size.
2. Under **Trusted sources**, add the droplet.
3. Copy the **connection string** and set it in `.env`. Keep `sslmode=require`. `dotenv COMPOSE_PROFILES=DATABASE_URL=postgres://doadmin:<password>@<host>:25060/defaultdb?sslmode=require`
4. The `db` container then doesn't start. Backups come from DigitalOcean, and `scripts/backup.sh` isn't used.

7. Build and start

```
cd /opt/sokopay/deploy
docker compose build                # about 5 minutes the first time
docker compose up -d db redis      # skip "db" if using Managed PostgreSQL
docker compose run --rm web python manage.py migrate --noinput
docker compose up -d
docker compose ps                  # web (healthy), worker, beat, db, redis, caddy: all "running"
```

Check it from your computer. All three should load:

<code>https://test.sokopay.com.gh/healthz</code>	-> {"status": "ok", ...}
<code>https://test.sokopay.com.gh/readyz</code>	-> {"status": "ok"} (database and Redis reachable)
<code>https://test.sokopay.com.gh/legal/</code>	-> the legal pages, with the orange TEST ENVIRONMENT banner

The certificate appears within a minute of the first request. If it doesn't, see section 13.

7.1 Load the demo data

```
docker compose run --rm web python manage.py seed_demo > ~/demo-credentials.md
chmod 600 ~/demo-credentials.md
cat ~/demo-credentials.md
```

This creates the same people, shop, agents and activity as the demo guide (docs/SokoPay-Demo-Guide.pdf), with one difference. Because this server is on the internet, **every password, PIN and 2FA secret is random**, not the published demo values. The real ones exist only in `~/demo-credentials.md` on the droplet. Share them with testers through a password manager, never in chat.

The shop's merchant code is still **GRACEMTN**.

8. Daily operations during the test

Task	Command (in <code>/opt/sokopay/deploy</code>)
Status	<code>docker compose ps</code>
Live logs (all)	<code>docker compose logs -f --tail 100</code>
Web or API logs only	<code>docker compose logs -f web</code>
Restart everything	<code>docker compose restart</code>
Stop / start	<code>docker compose stop / docker compose up -d</code>
Django shell	<code>docker compose run --rm web python manage.py shell</code>
Back up the database now	<code>bash scripts/backup.sh</code> (keeps the last 14 in <code>deploy/backups/</code>)
Nightly backups at 02:00	<code>(crontab -l; echo "0 2 * * * cd /opt/sokopay/deploy && bash scripts/backup.sh") \ crontab -</code>
Disk space	<code>df -h</code> and <code>docker system df</code>

Health:

- The staff portal's **Health** page shows stuck payouts, failed payments, overdue disputes and safeguarding.
- DigitalOcean **Monitoring**, then **Create alert**: CPU above 80%, memory above 85%, disk above 80%.

Restore a backup (for example, after a bad test step):

```
docker compose stop web worker beat
docker compose exec -T db dropdb -U sokopay sokopay
docker compose exec -T db createdb -U sokopay sokopay
docker compose exec -T db pg_restore -U sokopay -d sokopay --no-owner < backups/<file>.dump
docker compose up -d
docker compose run --rm web python manage.py db_fingerprint # ledger sums to zero, no integrity breaks
```

9. Updating to a new version

On your computer:

```
bash deploy/scripts/package.sh
scp sokopay-deploy.tgz deploy@203.0.113.10:/opt/sokopay/
```

On the droplet:

```
cd /opt/sokopay && tar xzf sokopay-deploy.tgz && rm sokopay-deploy.tgz
cd deploy && bash scripts/update.sh
```

`update.sh` backs up the database, rebuilds the image, runs migrations, restarts and checks `/readyz`. **Your `.env` is not touched**, because the package never contains it.

10. Testing with the apps and portals

10.1 Phone apps pointed at the server

On your computer, build the APKs with the server's address. Because the server has a real HTTPS certificate, the apps work on **real phones**, not just the emulator.

```
API_BASE_URL=https://test.sokopay.com.gh/api/v1 bash mobile/scripts/build_apks.sh
```

The APKs land in `mobile/dist/`. Send them to testers, who open them on Android and allow installation. They're debug-signed and for testing only.

10.2 Sign-in codes

With `SMS_PROVIDER=console`, no SMS is sent. Find a tester's code on the droplet:

```
docker compose logs web | grep "SMS:console" | tail -5
```

For a pilot with real SMS, set `SMS_PROVIDER=hubtel` plus `HUBTEL_CLIENT_ID`, `HUBTEL_CLIENT_SECRET` and `SMS_SENDER_ID` in `.env`, then `docker compose up -d`.

10.3 Portals

- **Address:** `https://test.sokopay.com.gh/dashboard/`. Sign in with the phone and password from `demo-credentials.md`.
- **2FA:** add the account's 2FA secret to an authenticator app, or print the current code: `bash docker compose run --rm web python manage.py demo_2fa_code +233200000004`
- **Mock partner:** `https://test.sokopay.com.gh/dev/mock-partner/` approves MoMo prompts and payouts (demo guide, section 11). On the server it needs the **superuser** (Abena Admin) signed in with 2FA.

10.4 USSD

USSD test calls must include the secret from `.env` (`grep USSD_SHARED_SECRET .env`):

```
curl -s -X POST "https://test.sokopay.com.gh/api/v1/usss/callback?key=<USSD_SHARED_SECRET>" \
--data-urlencode sessionId=t1 --data-urlencode phoneNumber=+233244000201 --data-urlencode text=
```

A real USSD aggregator is given the same URL and secret.

10.5 Following the demo guide

Every walkthrough in `docs/SokoPay-Demo-Guide.pdf` works on the server with these changes:

- Use `https://test.sokopay.com.gh` instead of `http://127.0.0.1:8000`.
- Use the logins from `demo-credentials.md`.
- Read sign-in codes from `docker compose logs web`.

11. After the test: wipe everything

When testing is complete and nothing needs to be kept:

1. **(Optional) keep evidence:** export what you need from the portal first, such as the audit log CSV, regulatory figures and statements. Copy them to your computer: `bash scp deploy@203.0.113.10:/opt/sokopay/deploy/backups/<latest>.dump .` Only keep a copy if you need it; it contains the test data.
2. **Wipe the data on the server:** `bash cd /opt/sokopay/deploy && bash scripts/wipe.sh` Type `WIPE` to confirm. This deletes: - the PostgreSQL and Redis data, certificates, backups and nightly backup job; - `.env`, including every secret; - the built images.
3. **Delete the cloud resources** in the DigitalOcean control panel: - **Droplets**, then `sokopay-test`, then **Destroy**. This erases the disk. - **Backups and Snapshots:** delete any droplet backups or snapshots. - **Databases:** delete the managed cluster, if you used one. - **Networking:** delete the `sokopay-test` firewall and any DNS records you added. - **Monitoring:** delete the alert policies.
4. **On your computer:** delete `demo-credentials.md` and any downloaded dumps. Uninstall the test APKs from testers' phones.
5. **If any real credentials were used** (Hubtel, Firebase, Sentry), rotate or revoke them at the provider.

After step 3 nothing of the test remains at DigitalOcean.

12. Moving to real production later

The pilot proves the platform. Real production differs in five ways:

Area	Pilot (this guide)	Production
Settings	<code>config.settings.staging</code> (mock partner allowed)	<code>config.settings.prod</code> : refuses to start with any mock, and needs a configured and certified Korba or Nsano (<code>docs/RAILS-INTEGRATION.md</code>)
Data location	DigitalOcean London or Frankfurt	Hosting approved by the Bank of Ghana and the Data Protection Commission (in Ghana, or AWS Cape Town per <code>infra/</code> if approved)
Database	PostgreSQL on the droplet	Managed PostgreSQL with automatic failover, point-in-time recovery, and a restore drill (<code>docs/ENGINEERING-OPERATIONS.md</code>)
Servers	One droplet	At least two app servers behind a load balancer, separate worker, monitoring alarms (<code>infra/monitoring.tf</code>)

Area	Pilot (this guide)	Production
Data	Demo only	Real customers, so the legal documents must be signed off first (docs/legal)

13. Troubleshooting

Symptom	Fix
<code>https://...</code> shows a certificate error	DNS must point at the droplet, and ports 80 and 443 must be open in both firewalls. See <code>docker compose logs caddy</code> .
Bad Request (400)	The domain is missing from <code>ALLOWED_HOSTS</code> in <code>.env</code> . Restart with <code>docker compose up -d</code> .
CSRF verification failed on portal forms	<code>CSRF_TRUSTED_ORIGINS</code> must be <code>https://<your domain></code> .
<code>web</code> keeps restarting	<code>docker compose logs web</code> . Usually a missing <code>.env</code> value (<code>FIELD_ENCRYPTION_KEY</code> , <code>SECRET_KEY</code>) or a wrong <code>DATABASE_URL</code> .
<code>readyz</code> says unavailable	The database or Redis isn't reachable: <code>docker compose ps</code> , then <code>docker compose logs db redis</code> .
Image build runs out of memory	Make sure the 2 GB swap from section 3 is on (<code>swapon --show</code>), or use the 8 GB droplet.
Apps can't connect	The APK must be built with <code>API_BASE_URL=https://<domain>/api/v1</code> . Plain <code>http://</code> addresses are blocked except the local emulator address.
A MoMo payment stays "pending"	Approve it on <code>/dev/mock-partner/</code> as the superuser.
"Demo data is already loaded"	The seed only runs on an empty database. Wipe and start again (section 11, steps 1–2, then section 7).